

**EXERCISE 8.14**

Write a property to verify the behavior of `List.sorted` (API docs link: <http://mng.bz/Pz86>), which you can use to sort (among other things) a `List[Int]`.⁷ For instance, `List(2, 1, 3).sorted` is equal to `List(1, 2, 3)`.

8.4.2 Writing a test suite for parallel computations

Recall that in chapter 7 we discovered laws that should hold for our parallel computations. Can we express these laws with our library? The first “law” we looked at was actually a particular test case:

```
map(unit(1))(_ + 1) == unit(2)
```

We certainly can express this, but the result is somewhat ugly.⁸

```
val ES: ExecutorService = Executors.newCachedThreadPool
val p1 = Prop.forAll(Gen.unit(Par.unit(1)))(i =>
  Par.map(i)(_ + 1)(ES).get == Par.unit(2)(ES).get)
```

We’ve expressed the test, but it’s verbose, cluttered, and the *idea* of the test is obscured by details that aren’t really relevant here. Notice that this isn’t a question of the API being expressive enough—yes, we can express what we want, but a combination of missing helper functions and poor syntax obscures the intent.

PROVING PROPERTIES

Let’s improve on this. Our first observation is that `forAll` is a bit too general for this test case. We aren’t varying the input to this test, we just have a hardcoded example. Hardcoded examples should be just as convenient to write as in a traditional unit testing library. Let’s introduce a combinator for it (on the `Prop` companion object):

```
def check(p: => Boolean): Prop
```

How would we implement this? One possible way is to use `forAll`:

```
def check(p: => Boolean): Prop = {
  lazy val result = p
  forAll(unit(()))(_ => result)
}
```

← **Note that we are non-strict here.**

← **Result is memoized to avoid recomputation.**

But this doesn’t seem quite right. We’re providing a unit generator that only generates a single value, and then we’re proceeding to ignore that value just to drive the evaluation of the given `Boolean`.

Even though we memoize the result so that it’s not evaluated more than once, the test runner will still generate multiple test cases and test the `Boolean` multiple times.

⁷ `sorted` takes an *implicit* `Ordering` for the elements of the list, to control the sorting strategy.

⁸ This is assuming our representation of `Par[A]` that’s just an alias for the function type `ExecutorService => Future[A]`.

For example, if we say `run(check(true))`, this will test the property 100 times and print “OK, passed 100 tests.” But checking a property that is always true 100 times is a terrible waste of effort. What we need is a new primitive.

Remember, the representation of `Prop` that we have so far is just a function of type `(MaxSize, TestCases, RNG) => Result`, where `Result` is either `Passed` or `Falsified`. A simple implementation of a `check` primitive is to construct a `Prop` that ignores the number of test cases:

```
def check(p: => Boolean): Prop = Prop { (_, _, _) =>
  if (p) Passed else Falsified("()", 0)
}
```

This is certainly better than using `forall`, but `run(check(true))` will still *print* “passed 100 tests” even though it only tests the property once. It’s not really true that such a property has “passed” in the sense that it remains unfalsified after a number of tests. It is *proved* after just one test. It seems that we want a new kind of `Result`:

```
case object Proved extends Result
```

Then we can just return `Proved` instead of `Passed` in a property created by `check`. We’ll need to modify the test runner to take this case into account.

Listing 8.6 Using `run` to return a `Proved` object

```
def run(p: Prop,
        maxSize: Int = 100,
        testCases: Int = 100,
        rng: RNG = RNG.Simple(System.currentTimeMillis)): Unit =
  p.run(maxSize, testCases, rng) match {
    case Falsified(msg, n) =>
      println(s"! Falsified after $n passed tests:\n $msg")
    case Passed =>
      println(s"+ OK, passed $testCases tests.")
    case Proved =>
      println(s"+ OK, proved property.")
  }
```

We also have to modify our implementations of `Prop` combinators like `&&`. These changes are quite trivial, since such combinators don’t need to distinguish between `Passed` and `Proved` results.



EXERCISE 8.15

Hard: A `check` property is easy to prove conclusively because the test just involves evaluating the `Boolean` argument. But some `forall` properties can be proved as well. For instance, if the domain of the property is `Boolean`, then there are really only two cases to test. If a property `forall(p)` passes for both `p(true)` and `p(false)`, then it is proved. Some domains (like `Boolean` and `Byte`) are so small that they can be exhaustively checked. And with sized generators, even infinite domains can be exhaustively

checked up to the maximum size. Automated testing is very useful, but it's even better if we can *automatically prove our code correct*. Modify our library to incorporate this kind of exhaustive checking of finite domains and sized generators. This is less of an exercise and more of an extensive, open-ended design project.

TESTING PAR

Getting back to proving the property that `Par.map(Par.unit(1))(_ + 1)` is equal to `Par.unit(2)`, we can use our new `Prop.check` primitive to express this in a way that doesn't obscure the intent:

```
val p2 = Prop.check {
  val p = Par.map(Par.unit(1))(_ + 1)
  val p2 = Par.unit(2)
  p(ES).get == p2(ES).get
}
```

This is now pretty clear. But can we do something about the `p(ES).get` and `p2(ES).get` noise? There's something rather unsatisfying about it. For one, we're forcing this code to be aware of the internal implementation details of `Par` simply to compare two `Par` values for equality. One improvement is to *lift* the equality comparison into `Par` using `map2`, which means we only have to run a single `Par` at the end to get our result:

```
def equal[A](p: Par[A], p2: Par[A]): Par[Boolean] =
  Par.map2(p, p2)(_ == _)

val p3 = check {
  equal(
    Par.map(Par.unit(1))(_ + 1),
    Par.unit(2)
  )(ES).get
}
```

This is a bit nicer than having to run each side separately. But while we're at it, why don't we move the running of `Par` out into a separate function, `forAllPar`. This also gives us a good place to insert variation across different parallel strategies, without it cluttering up the property we're specifying:

```
val S = weighted(
  choose(1, 4).map(Executors.newFixedThreadPool) -> .75,
  unit(Executors.newCachedThreadPool) -> .25)

def forAllPar[A](g: Gen[A])(f: A => Par[Boolean]): Prop =
  forAll(S.map2(g)((_, _))) { case (s, a) => f(a)(s).get }
```

*a -> b
is syntactic
sugar for
(a, b).*

*This generator
creates a fixed
thread pool
executor 75% of
the time and an
unbounded one
25% of the time.*

`S.map2(g)((_,_))` is a rather noisy way of combining two generators to produce a pair of their outputs. Let's quickly introduce a combinator to clean that up:⁹

```
def **[B](g: Gen[B]): Gen[(A,B)] =
  (this map2 g)((_,_))
```

Much nicer:

```
def forAllPar[A](g: Gen[A])(f: A => Par[Boolean]): Prop =
  forAll(S ** g) { case (s,a) => f(a)(s).get }
```

We can even introduce `**` as a pattern using custom extractors (<http://mng.bz/4pUc>), which lets us write this:

```
def forAllPar[A](g: Gen[A])(f: A => Par[Boolean]): Prop =
  forAll(S ** g) { case s ** a => f(a)(s).get }
```

This syntax works nicely when tupling up multiple generators—when pattern matching, we don't have to nest parentheses like using the tuple pattern directly would require. To enable `**` as a pattern, we define an object called `**` with an `unapply` function:

```
object ** {
  def unapply[A,B](p: (A,B)) = Some(p)
}
```

See the custom extractors documentation for more details on this technique.

So `S` is a `Gen[ExecutorService]` that will vary over fixed-size thread pools from 1–4 threads, and also consider an unbounded thread pool. And now our property looks a lot cleaner:¹⁰

```
val p2 = checkPar {
  equal (
    Par.map(Par.unit(1))(_ + 1),
    Par.unit(2)
  )
}
```

These might seem like minor changes, but this sort of factoring and cleanup can greatly improve the usability of our library, and the helper functions we've written make the properties easier to read and more pleasant to write. You may want to add a `forAllPar` version for sized generators as well.

Let's look at some other properties from chapter 7. Recall that we generalized our test case:

```
map(unit(x))(f) == unit(f(x))
```

⁹ Calling this `**` is actually appropriate, since this function is taking the *product* of two generators, in the sense we discussed in chapter 3.

¹⁰ We can't use the standard Java/Scala `equals` method, or the `==` method in Scala (which delegates to the `equals` method), since that method returns a `Boolean` directly, and we need to return a `Par[Boolean]`. Some infix syntax for `equal` might be nice. See the answer file for chapter 7 for an example of how to do this.